```
            break;
          }
        }
      }
      return newHeight;
};
```

The stacked area chart in figure 4.24 is already complex. To make it a proper streamgraph, the stacks need to alternate. This requires a more complicated stacking function.

Listing 4.15 A stacking function that alternates vertical position of area drawn

```
…
var movieArea = d3.svg.area().x(function(d) {
                    return xScale(d.day)
            })
.y(function(d) {
                    return yScale(alternatingStacking(d,x,"top"))
            })
.y0(function(d) {
                    return yScale(alternatingStacking(d,x,"bottom"));
            }).interpolate("basis");
…
function alternatingStacking(incomingData,incomingAttribute,topBottom)
{
var newHeight = 0;
var skip = true;
for (x in incomingData) {
    if (x != "day") {
        if (x == "movie1" || skip == false) {
            newHeight += parseInt(incomingData[x]);
          if (x == incomingAttribute) {
            break;
          }
          if (skip == false) {
            skip = true;
          } else {
            n%2 == 0 ? skip = false : skip = true;
          }
        } else {
            skip = false;
        }
      }
    }
    if(topBottom == "bottom") {
        newHeight = -newHeight;
    }
    if (n > 1 && n%2 == 1 && topBottom == "bottom") {
        newHeight = 0;
    }
    if (n > 1 && n%2 == 0 && topBottom == "top") {
        newHeight = 0;
    }
    return newHeight;
};
```

We can create whatever complex accessor function we want for our generators.

We need the data, and we also need to know whether we're drawing the top or bottom of the area, which alternates as we move through the dataset.

Always skips day, because that's just our x position

Skips the first movie (our center), and then skips every other movie to get the alternating pattern

Stops when we reach this movie, which gives us the baseline

The height is negative for areas on the bottom side of the streamgraph, and positive for those on the top side.